Le langage R : quelques fonctions utiles pour l'analyse statistique des données

R est un langage qui permet d'exécuter des commandes pour l'analyse statistique et la visualisation des données. C'est un programme gratuit qu'on peut le télécharger sur le site : http://www.r-project.org/. Une documentation complète y est aussi téléchargeable.

1. Lecture des données

Commencez par créer un répertoire où vous mettrez vos fichiers de données (on utilisera pour illustrer la suite des commandes et analyses le fichier *exemple1.xls*).
Lorsque vous ouvrez une nouvelle session R changer le répertoire courent à partir du menu pour se mettre sur le répertoire où se trouvent les fichiers de données.
Transformer les fichiers est au format Excel en fichier au format csv, créer une copie (en gardant le même nom) au format .csv (séparateur pont virgule).
Pour lire le fichier dans un objet (nommé X) utiliser la commande :
`X=read.csv("exemple1.csv",sep=";",dec=",",header=T)`
`dec` correspond à la notation de la décimale dans le fichier excel d'origine (. ou ,).
X contient 4 variables nommées V1, V2, V3 et V4 où les deux premières (V1, V2) sont qualitatives et les deux suivantes (V3,V4) quantitatives.
`ls()` affiche la liste des objets de l'espace de travail.
`dir()` affiche les fichiers dans le répertoire courant.
`str(X)` : affiche la liste des variables de X et leur type (numérique, facteur,..)

2. Manipulation de données

Le l'objet X (dataframe) se manipule comme une matrice :
`X[,2]` affiche la deuxième colonne de X ; `X[,1:2]` affiche les colonnes 1 et 2 de X
`X[,c(1,3)]` affiche les colonnes 1 et 3 de X
`X[3,5]` affiche la 3ème ligne de la 5ème colonne de X
`X[X>3]` affiche les valeurs supérieures à 3 dans le tableau X
`X$V1` affiche la variable V1 du tableau X
`X$V1[3]` affiche la 3ème valeur de la variable V1
`X$V3[3]=10` modifie la 3ème valeur de la variable V3 et lui donne la valeur 10
On peut transformer le type d'une variable par les commandes :
`X$V1<-as.factor(X$V1)` (idem avec `as.numeric, ordered`, ..)
`which(X$V4>5)` : donne les lignes pour lesquelles la valeur de V4>5
Recoder une variable : `X$V2[X$V2<6]=0 ; X$V2[X$V2>=6]=1`

3. Statistiques descriptives :

a- <u>pour les variables qualitatives</u> (en classes)
Calcul les effectifs (table de fréquence): `tab=table(X$V2)`
Dessiner un diagramme en baton: `barplot(tab)`
Dessiner un diagramme en secteurs (camembert): `pie(tab)`
b- <u>pour les variables quantitatives</u>
Dessiner un histogramme : `hist(X$V3), hist(X$V3,col="grey",breaks=10)`
Dessiner un qqplot pour tester la Normalité : `qqnorm(X$V3)` puis `qqline(X$V3)`
Dessiner un box-plot : `boxplot(X$V3)` ou selon des classes définies par une autre variable : `boxplot(X$V3~X$V1)` ou par deux autres variables: `boxplot(X$V3~X$V1+X$V2)`
Dessiner un strip-chart : `stripchart(X$V3)`
Calcul de moyenne, mediane, quartiles : `summary(X$V3)`
Moyenne : `mean(X$V3)`; Mediane : `median(X$V3)`; Ecart-type : `sd(X$V3)`; Variance : `var(X$V3)`, si on veut les valeurs par groupe on utilise : `tapply(X$V3,X$V1,mean)`

Pour savoir si une variable suit une loi Normale
Test de Kolmogorov-Smirnov : `ks.test(X$V3,"pnorm",mean=mean(X$V3),sd=sd(X$V3))`
Test de Shapiro-Wilks : `shapiro.test(X$V3)`
Remarque : vous pouvez mettre plusieurs graphiques côte à côté en utilisant l'instruction : par(mfrow=c(1,2))
Pour sauvegarder un graphique vous pouvez (bouton droit de la souris) soit faire copier-coller dans word ou sauvegarder en fichier au format postscript.

3. Aalyses bivariées (relations entre deux variables)
a- pour les variables qualitatives (en classes)
Obtenir la table croisées de effectifs : `tab=table(X$V1,X$V2)`
Test chi-deux: `chisq.test(tab)`
Test exact de Fisher : `fisher.test(tab)` Graphique: `mosaicplot(tab)`
b- pour une variable quantitative et une qualitative
Test de Student pour échantillons indépendants : `t.test(X$V3~X$V1,var.equal=TRUE)`
Comparer les variances : `var.test(X$V3~X$V1)`
Test de Student pour échantillons appariés : `t.test(X$V3,X$V3,paired=TRUE)`
Test non-paramétrique de Wilcoxon (Mann Whitney): `wilcox.test(X$V4~X$V1)`
Test de Wilcoxon pour échantillon appariés : `wilcox.test(X$V1,X$V2,paired=TRUE)`
Si on veut tester si deux variables ont la même distribution :
Test de Kolmogorov-Smirnov : `ks.test(X$V3~X$V1)`

c- pour deux variable quantitatives
Dessiner un diagramme de dispersion : `plot(X$V4~X$V3)` ou `sunflowerplot()`
Plusieurs graphiques pour chaque valeur : `coplot(X$V4~X$V3|X$V1)`
Calculer un coefficient de corrélation entre deux variables:
Coefficient de Pearson : `cor(X$V3,X$V4)` et avec le test: `cor.test(X$V3,X$V4)`
Coefficient de Spearman : `cor(X$V3,X$V4,method="spearman")`
Droite de régression entre deux variables : `m=lm(X$V4~X$V3)`
Dessiner la droite de régression : `abline(m)`

6. Etudier la relation entre une variable réponse et plusieurs variables explicatives
a- La variable réponse est quantitative et les variables explicatives sont qualitatives (facteurs) : ***analyse de variance*** (anova)
si on note `Y` la variable réponse et `A,B,C`.. les facteurs (transformer par `as.factor`)
pour modèle factoriel complet : `av=aov(Y~A*B*C,data=X)` puis `summary(av)`
Validation graphique du modèle : `plot(av)`
Test de comparaison multiple (post-hoc) : `TukeyHSD(av,"A")`
Anova non parametrique : `kruskal.test(y~A)`
b- La variable réponse est quantitative et les variables explicatives sont quantitatives : ***régression multiple***
régression linéaire : `m=lm(y~x1+x2+x3,data=X)` puis `summary(m)` ou `anova(m)`
autres modèles (quadratique) `m=lm(Y~poly(x1,2)+x2^2+x1:x2)`
dessiner une courbe sur un graphe : `lines(x,a*x^2+b*x+c)`
sélection de modèle : `m=lm(Y~.)` puis `step(m)` (ou `stepAIC` du package MASS)
Tous les graphiques 2 à 2: `pairs(X)`
c- la variable réponse est qualitative : ***régression logistique***
`rg=glm(V1~.,family=binomial(logit),X)` puis `summary(rg)`
avoir les rapport de côte et intervalle confiance : `OR=exp(coef(rg)) ; CI=exp(confint(rg))`
sélection de modèle : `st=step(rg,direction="both")` puis `summary(st)`

7. Analyses factorielles et de classification

7.1 Analyse factorielles

a- Analyse en Composantes Principales (ACP)

`pca=princomp(X)` puis `summary(pca)`

`pca$loadings`: avoir les coefficients des axes

Projections des observations et variables: `biplot(pca,xlabs=X$lab)`

Rajouter des axes au centre: `abline(h=0)` et `abline(v=0)`

b- Analyse factorielle des correspondance (AFC ou CA en anglais)

Chargez d'abord la library MASS

`ca=corresp(X,nf=2)` puis `summary(ca)`

Projections des observations (lignes) et variables (colonnes): `biplot(ca)`

Correspondence multiple: `mc=mca(X,abbrev=T)` puis `plot(mc)`

c-Analyse discriminante linéaire (ADL ou LDA en anglais)

`ld=lda(X,grouping=X$V1)`: la variable V1 contient l'indicateur des groups

`plot(ld); pred1=predict(ld,X)$class` : donne le vecteur des classes prédites

`table(X$V1,pred1)`: permet de voir le taux d'observations bien classées.

`qd=qda(X,grouping=X$V1)`: discrimination quadratique

7.2. Classification

a- Hierarchical classification (HCA):

Calculer la matrice de distance matrix: `D=dist(X, method= "euclidian")`

voir help for other distance measures (par défaut euclidian distance est utilisée).

Créer les clusters: `cl=hclust(D, method="ward.D")`

voir help for d'autres methodes (average, simple,..)

Visualiser l'abre de classification (dendrogramme) `plclust(cl,labels=X$lab)`

Obtenir de sgroupes en coupant l'arbre: `cutree(cl,k=3)` de sorte avoir k groups ou à une distance h donnée `cutree(cl,h=2)`

Évaluer la qualité de l'arbre par rapport aux distances: `D1=cophenetic(cl)` puis `cor(D,D1)` ou `Mantel.rtest(D,D1,nerept=99)` (package ade4)

Evaluer par bootstrap: (package pvclust): `hc=pvclust(X)` puis `plot(hc)`

b- Méthode K-means

`km=kmeans(X, centers, nstart, algorithm="Lloyd")`

`centers` est le nombre de groupes souhaités. Voir help pour d'autres choix possibles d'algorithmes (par défault algorithme de Hartigan et Wong).

Visualiser les groupes: `plot(X,col=km$cluster)` vous pouvez ajouter à ce graphique les centres des groupes indiqués par le signe '+' par:

`points(km$centers,col=1:2,pch=8)`

c-Autres options

pour la distance generalise de gower (qui se calcule sur un mélange de variables qualitatives et quantitative) on peut utilise le package `StatMatch` et la function `gower.dist(X)`.